

Digital Distribution Operating System (DDOS) - Sarthi Wholesale

Developer Specifications & Customization Document

This document outlines the system requirements and development plan for the Sarthi Wholesale DDOS project, mapped against the existing `egrocer` architecture. It is divided actor-wise, highlighting what is **Already Implemented (Existing)** and what needs to be **Built (New/Customization)**.

1. Super Admin

The Super Admin is the control tower for the master catalog and ecosystem configuration.

● Already Implemented

- **Master Product System:** Create categories, sub-categories, products, units, weights.
- **Brand Management:** Basic brand creation.
- **Admin Authentication:** Login, roles, permissions.

● To Build (Customizations)

- **Territory Control & Geo-Fencing:**
 - Ability to map Brands to specific Distributors.
 - Add "Overlap Allowed" toggle when creating/mapping a brand.
 - Map specific geographic areas (Zones/Pincodes) to distributors for specific brands.
- **Tally Export Config:** Configure Tally integration/export formats.

🔧 APIs Needed (Super Admin)

Existing:

- `POST /api/admin/login`
- `GET/POST /api/admin/products`
- `GET/POST /api/admin/brands`

New:

- `POST /api/admin/brand-mappings` (Assign brand to distributor with overlap flag)
 - `POST /api/admin/geo-fences` (Map areas/zones to distributors)
-

2. Distributor (Seller)

The Distributor manages local stock, order processing, and dispatch.

● Already Implemented

- **Distributor Auth & Profile:** Seller registration, login.
- **Product Control:** Activate/deactivate products, set local selling price, MRP, and stock quantity.
- **Basic Order Management:** View orders, change statuses.
- **Wallet/Transactions:** Seller wallet transactions.

● To Build (Customizations)

- **Order Cutoff Logic:** Auto-tag orders placed before 3:00 PM as "Next Day Delivery" and after 3:00 PM as "Day After Delivery".
- **Zone Grouping:** Group orders by Delivery Zone/Area showing total weight and value.
- **Vehicle & Driver Assignment:**
 - Create Vehicles (with capacity in kg).
 - Smart Flow: Assign driver and orders/zones to a vehicle with weight validation (Assigned Weight $\leq$ Vehicle Capacity).
- **Loading Slip Generation:** Generate PDF loading slips (Item-wise qty, total items, shop names, total weight).
- **Dispatch Flow:** Lock order editing after dispatch. Statuses: `Dispatch Ready` -> `Dispatched`.
- **Tally Export Module:** Export Sales Invoices, Credit Notes, and Payments.
- **Credit Note System (Auto):** Auto-generate credit notes upon "Short Delivery" by the driver.
- **Salesman Limits:** Define maximum discount permissions for salesmen.
- **Subscription/Billing:** Subscription page showing saved amount and billing overview.

🔧 APIs Needed (Distributor)

Existing:

- `GET /api/seller/orders`
- `PUT /api/seller/orders/{id}/status`
- `GET /api/seller/products`

New:

- `GET /api/seller/orders/grouped-by-zone` (Fetch orders grouped by delivery zone for planning)
- `POST /api/seller/vehicles` (CRUD for vehicles)
- `POST /api/seller/dispatch/assign` (Assign orders to driver/vehicle, validate weight)
- `GET /api/seller/loading-slip/{dispatch_id}` (Generate PDF)
- `GET /api/seller/exports/tally` (Generate Tally export data)
- `POST /api/seller/salesmen/limits` (Set max discount for salesmen)

3. Salesman (New App)

A completely new actor. The Salesman focuses on retailer onboarding, relationship management, and assisted ordering.

🟢 Already Implemented

- *None. This is a completely new actor/model that needs to be created.*

🔴 To Build (Customizations)

- **Auth & Role System:** Salesman Model, OTP Login, Security App Lock (FaceID/Fingerprint).
- **Daily Plan / Dashboard:** Show assigned retailers, pending verifications, and daily targets.
- **Retailer Onboarding (Verification):**
 - Capture GPS location, Storefront Photo, Shop Name, Party Name (Tally sync), GST, Address.
 - Approve retailer to "Active" status.
- **Assisted Order:**
 - Place orders on behalf of the retailer.
 - Edit product prices during order placement (within Distributor-defined limits).

- **QR Code Scanning:** Scan Retailer QR code for quick identification and order placement.

🔧 APIs Needed (Salesman)

New:

- `POST /api/salesman/login` (OTP based)
 - `GET /api/salesman/dashboard` (Daily plan, targets)
 - `GET /api/salesman/retailers` (List mapped retailers)
 - `POST /api/salesman/retailers/onboard` (Upload GPS, Photos, Details)
 - `POST /api/salesman/retailers/{id}/verify` (Approve retailer)
 - `POST /api/salesman/orders` (Assisted order creation with price overrides)
 - `POST /api/salesman/scan-qr` (Identify retailer by QR)
-

4. Retailer (Buyer / HORECA)

The end customer ordering from the distributor.

🟢 Already Implemented

- **Auth:** OTP Login/Signup.
- **Catalog & Cart:** Browse products, Add to Cart.
- **Checkout:** Address selection, place order.
- **Order History:** View previous orders.

🔴 To Build (Customizations)

- **Geo-Locked Catalog:** Retailer sees ONLY products/prices from their mapped Distributor.
- **Multi-Vendor Cart & Checkout:**
 - Group cart by distributor.
 - Minimum order validation per distributor.
 - Allow partial checkout if some distributors don't meet minimums.
- **Slab Pricing & Schemes:**
 - Display quantity stepper with dynamic pricing based on slabs (e.g., 1-10: ₹50, 11-50: ₹48).
 - Buy X Get Y schemes, Group slab discounts.
- **Retailer Ledger (Account Statement):** Running balance showing Invoices (+) and Payments (-).
- **QR Code System:** Auto-generate a unique QR code for the retailer profile after approval.
- **Order Modifications:** Reschedule or cancel orders (if shop closed/unavailable).

🔧 APIs Needed (Retailer)

Existing:

- `POST /api/login`
- `GET /api/products` (Needs modification to filter by mapped distributor)
- `GET/POST /api/cart`
- `POST /api/checkout`
- `GET /api/orders`

New:

- `GET /api/retailer/ledger` (Fetch account statements and running balance)
- `GET /api/retailer/qr-code` (Fetch unique QR code)
- `PUT /api/orders/{id}/reschedule`

- `PUT /api/orders/{id}/cancel`
-

5. Driver (Delivery Boy)

Executes the delivery, tracks partial deliveries, and collects payments securely.

● Already Implemented

- **Driver Auth:** Delivery boy login.
- **Order Execution:** List assigned orders, mark delivered.
- **Live Tracking:** Basic location tracking.

● To Build (Customizations)

- **Start Day & Route List:** View total stops, expected cash collection, and clustered route mapping (nearest neighbor logic).
- **Execution Flow (Zero Error Zone):**
 - **Partial Delivery:** Driver inputs delivered qty ($\text{Delivered} \leq \text{Ordered}$). Auto-detect shortage and automatically reduce the final amount.
 - **Exception Handling:** Mark "Shop Closed", "Refused", or "Damage" (with optional photo).
- **Payment Collection:**
 - Auto-calculate final amount.
 - Split collection between Cash and UPI.
 - Upload UPI screenshot/proof.
 - In-App OTP confirmation from Retailer to complete delivery.
- **End of Day Settlement:** Daily summary comparing Expected Cash vs Collected Cash/UPI. Lock after submission. Offline mode sync support.

🔧 APIs Needed (Driver)

Existing:

- `POST /api/delivery-boy/login`
- `GET /api/delivery-boy/orders` (Needs update to return clustered route)
- `PUT /api/delivery-boy/orders/{id}/status`

New:

- `POST /api/delivery-boy/trip/start`
 - `POST /api/delivery-boy/orders/{id}/execute` (Submit partial qty, shortages, damage photos)
 - `POST /api/delivery-boy/orders/{id}/payment` (Submit Cash/UPI split, UPI photo)
 - `POST /api/delivery-boy/orders/{id}/verify-otp` (Retailer OTP for delivery confirmation)
 - `GET /api/delivery-boy/settlement` (Fetch End of Day summary)
 - `POST /api/delivery-boy/settlement/submit` (Submit final collection to Admin/Distributor)
-

Technical Summary for Implementation

1. Database Schema Additions:

- `Vehicles` table.
- `LoadingSlips` table.
- `Salesmen` table (and associated Auth guards).

- `RetailerLedgers` table.
- `SlabPrices` & `Schemes` tables.
- `TallyExports` logging table.

2. Core Logic Upgrades:

- Modify the main Product querying scope to strictly join on Geo-fenced Distributor boundaries.
- Overhaul the Cart logic to group by Distributor ID.
- Implement the `OrderCutoffJob` (Cron/Schedule) for timestamp-based delivery date tagging.
- Introduce event listeners for `OrderPartiallyDelivered` to auto-trigger `CreditNote` generation.